CYBERSECURITY HOME LAB REPORT

Monitoring the Windows Downloads Folder with Wazuh FIM

Real-time, who-data detection of file creation, modification and deletion in a user-writable folder, and the limits of what file integrity monitoring can see

Day 9 Extension — Wazuh SOC Analyst Home Lab, 14-Day Intensive Program

Author	Aigbokhaode Hope Imomoh
Role	SOC Analyst Trainee
Date	12 November 2026
Version	1.0
Environment	Oracle VirtualBox: Windows 10 endpoint with Wazuh agent (ID 006) and Ubuntu server running the Wazuh manager and dashboard
Evidence period	Screenshots captured on 12 November 2026, 10:31 to 11:47

Evidence statement
All results come from an isolated home lab. Screenshots are cropped and enlarged for presentation only; originals are retained unchanged. Evidence gaps and unresolved items are stated openly in Sections 8.3 and 16.3 and in Chapter 10. This report extends the Day 9 report on File Integrity Monitoring.

Table of Contents

1. Executive Summary 7

2. Introduction and Objectives 9
- 2.1 Background and motivation 9
- 2.2 Objectives 9
- 2.3 Scope 9
- 2.4 Measurable success criteria 9

3. Technical Background 10
- 3.1 File Integrity Monitoring (FIM) 10
- 3.2 Hashes 10
- 3.3 Scheduled, real-time and who-data modes 10
- 3.4 From event to dashboard: six separate stages 10
- 3.5 Rules and MITRE ATT&CK mapping 11

4. Lab Environment and Architecture 12
- 4.1 Components 12
- 4.2 Architecture and data flow 12

5. Prerequisites and Safety 13
- 5.1 Prerequisites 13
- 5.2 Safety measures used 13
- 5.3 Restoring the lab if something goes wrong 13

6. Step-by-Step Implementation 14
- 6.1 Step 1 — Find the exact Downloads path 14
- 6.2 Step 2 — Back up and edit the agent configuration 14
- 6.3 Step 3 — Restart the agent and check the service 15
- 6.4 Step 4 — Verify that FIM started (agent log) 15
- 6.5 Step 5 — First dashboard search: "No results" 16
- 6.6 Step 6 — Test 1: create a file 17
- 6.7 Step 7 — Test 2: edit and delete in the same second 20
- 6.8 Step 8 — Test 3: a clean add, modify, delete lifecycle 20

7. Configuration and Detection Logic 27
- 7.1 The configuration change (the only one made) 27
- 7.2 Original, changed and recommended 27
- 7.3 Rules that fired 27
- 7.4 Fields used for detection and investigation 28
- 7.5 Dashboard searches 28

8. Testing and Results 29
- 8.1 Test scenarios 29
- 8.2 File timeline (required deliverable) 29
- 8.3 What succeeded, failed and remains untested 30

9. Evidence and Screenshot Analysis 30
- 9.1 Evidence handling 30
- 9.2 Background alerts seen alongside the test 30
- 9.3 What each evidence group proves 30

10. Troubleshooting and Root-Cause Analysis 31

11. Security Findings and Risk Analysis 31
- 11.1 Findings 31
- 11.2 Analyst notes: the questions an analyst asks 32
- 11.3 Why context matters 32

12. Mitigation and Remediation 33
- 12.1 Implemented in the lab 33
- 12.2 Recommendations (not implemented) 33

13. What I Learned 34

14. Practical Knowledge and Competencies 34
- 14.1 Suggested next exercises 35

15. Reproduction Guide 36
- 15.1 Initial state 36
- 15.2 Setup 36

15.3 Test activities 36
15.4 Verification and evidence collection 36
15.5 Troubleshooting 37
15.6 Cleanup and rollback (documented, not executed in this lab) 37
16. Limitations and Future Improvements 37
16.1 Limits of the experiment 37
16.2 Timestamp and display anomalies 37
16.3 Evidence gaps 38
16.4 False positives and false negatives 38
16.5 Future improvements 38
17. Conclusion 38
Appendix A — Command History 38
Appendix B — Configuration and Log Excerpts 39
Appendix C — Evidence Register 39
Appendix D — Glossary 40
Appendix E — References 40

List of Figures

Figure 1 — The six stages between a file change and an analyst reading the alert. 10
Figure 2 — Laboratory architecture and data flow. Drawn from the environment evidenced in the screenshots. 12
Figure 3 — Backup command, Notepad open on ossec.conf, and the new Downloads line beneath the Day 9 FIM_Test line. Cropped to the relevant window. 15
Figure 4 — The same session after the edit: service restart and Running status. 15
Figure 5 — Agent log output confirming the Downloads path, real-time monitoring and the who-data engine. Transcription follows. 16
Figure 6 — Dashboard Events search for syscheck.path: *downloads* returning no results. Photograph of the screen, straightened and cropped; browser tabs excluded. Screen clock reads Nov 12 10:46. 17
Figure 7 — Events list: c:\users\administrator\downloads\wazuh_test.txt, event added, at the top; older registry alerts below. 17
Figure 8 — Creation alert: agent 006, DESKTOP-T6T1N2V, decoder syscheck_new_entry, full_log "File ... wazuh_test.txt' added, Mode: whodata". 18
Figure 9 — Creation alert: rule description "File added to the system.", firedtimes 1, groups ossec, syscheck, syscheck_entry_added, syscheck_file. 18
Figure 10 — Creation alert: process powershell.exe PID 6324, user Administrator (SID ending -500), event added, mode whodata, MD5 ac33b4936bf8ec6ec3286472f6880a56. 19
Figure 11 — Creation alert: path, SHA-1 bd90186c..., SHA-256 06dd26e2..., size 32 bytes, owner group Administrators. 19
Figure 12 — The edit and delete commands with their identical Get-Date outputs (11:19:33 AM). Enlarged crop of the PowerShell window; no error is displayed. 20
Figure 13 — Events list showing the full set of file events: wazuh_test2.txt deleted, modified, added; wazuh_test.txt deleted and added; the 10:36:32 warning row; registry noise below. 71 hits. 21
Figure 14 — wazuh_test2.txt creation: decoder syscheck_new_entry, full_log "added, Mode: whodata", alert ID 1794479230.886952. 21
Figure 15 — Creation alert: rule 554, level 5, PCI DSS 11.5, NIST SI.7; process powershell.exe PID 6324, user SID ending -500. 22
Figure 16 — Creation alert: user Administrator, event added, MD5 ac33b4936bf8ec6ec3286472f6880a56, mode whodata, SHA-1 bd90186c..., path wazuh_test2.txt. 22
Figure 17 — Modification alert: decoder syscheck_integrity_changed; changed attributes size, mtime, md5, sha1, sha256; "Size changed from '32' to '66'". 23
Figure 18 — Modification alert: rule 550, level 7, groups include syscheck_entry_modified, MITRE T1565.001 (Stored Data Manipulation, tactic Impact). 23
Figure 19 — Modification alert: same user and process (PID 6324); MD5 before ac33b4936bf8ec6ec3286472f6880a56, after 8797714049c5aeca57053737e8db1abd. 24
Figure 20 — Modification alert: SHA-256 before 06dd26e2..., size 32 to 66, alert timestamp 11:28:20.669. 24
Figure 21 — Deletion alert: rule 553, level 7, description "File deleted.", groups include syscheck_entry_deleted, firedtimes 2. 25
Figure 22 — Deletion alert: MITRE tactics Defense Evasion and Impact; techniques File Deletion and Data Destruction; process PowerShell PID 6324. 25
Figure 23 — Deletion alert: event deleted, last-known MD5 8797714049c5aeca57053737e8db1abd, SHA-1 172764506c5d8000cd10813eb5e10df295b8dc71, SHA-256 7c342b5c.... 26
Figure 24 — Deletion alert: last-known SHA-256 7c342b5c10209ddf09afc4f8236e4ee0cd6e9f5cf0f87847a9ed24ed7a5dd024, size 66, alert timestamp 11:30:20.889. 26
Figure 25 — Timeline of Downloads FIM alerts on 12 November 2026 (alert timestamps, local time). 29
Figure 26 — Two rows from the creation alert of wazuh_test2.txt. Upper panel: mtime_after 12:27:09. Lower panel: alert timestamp 11:27:10.148. Panels cropped from separate screenshots and joined; nothing altered. 37

List of Tables

Table 1 — Summary of verified outcomes 7
Table 2 — Success criteria and result 9
Table 3 — FIM monitoring modes 10
Table 4 — Pipeline stages and where each was checked 11
Table 5 — Laboratory components (as evidenced by the screenshots) 12
Table 6 — Transcription of the log lines shown in the figure above 16
Table 7 — Evidence captured for the creation of wazuh_test.txt 19
Table 8 — Evidence captured for the modification of wazuh_test2.txt 24
Table 9 — Line-by-line explanation 27
Table 10 — Configuration status 27
Table 11 — FIM rules observed for the Downloads folder 27
Table 12 — Wazuh FIM fields used in this lab 28
Table 13 — Searches used 28
Table 14 — Test scenarios, expectations and outcomes 29
Table 15 — File timeline 29
Table 16 — Evidence assessment 30
Table 17 — Problems encountered 31
Table 18 — Findings and confidence 31
Table 19 — Controls actually implemented 33
Table 20 — Recommended improvements 33
Table 21 — Skills mapped to lab tasks 34
Table 22 — Commands to run and what to expect 36
Table 23 — Common problems 37
Table 24 — Commands executed 38
Table 25 — Alert IDs read from screenshots 39
Table 26 — Evidence register 39
Table 27 — Glossary 40

Abbreviations and Acronyms

Table — Abbreviations

Abbreviation	Meaning
ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
DQL	Dashboard query language
FIM	File integrity monitoring
MD5 / SHA	Message-digest and Secure Hash Algorithm fingerprints
PID	Process ID
SACL	System access control list
SID	Security identifier
SOC	Security operations centre
VM	Virtual machine

1. Executive Summary

This report documents an extension of Day 9 of the Wazuh SOC Analyst home-lab programme. Day 9 proved that Wazuh File Integrity Monitoring (FIM) can detect the creation, modification and deletion of files in a purpose-built test folder, `C:\FIM_Test`. This extension asks the harder, more realistic question: *can the same capability watch a folder where untrusted files actually arrive, the Windows `Downloads` folder, and can it still tell an analyst who changed a file and with which program?*

The security problem

A Downloads folder is where documents, installers and scripts from the internet land first. It is user-writable, busy and rarely reviewed, which makes it a common staging area for malicious files and for tampering. A security operations centre (SOC) needs to know **that** a file appeared, changed or vanished there, **who** did it, and **with which program**. By default Wazuh watches only the paths it is told about, so a Downloads folder is invisible until it is added to the agent configuration.

What was done

- A single `<directories>` line for `C:\Users\Administrator\Downloads` was added to the Wazuh agent configuration with real-time monitoring and who-data attribution. A backup of the configuration was taken first, and `report_changes` was deliberately left off to avoid storing copies of arbitrary downloaded files.
- The agent log confirmed that the new path was monitored and that the real-time and who-data engines had started (messages 6003, 6012 and 6019).
- Two controlled test files, `wazuh_test.txt` and `wazuh_test2.txt`, were created, edited and deleted from PowerShell. The second file produced a clean *added, modified, deleted* timeline with user, process and hashes; the first exposed a limitation, because an edit and a delete issued in the same second produced no "modified" alert.

Key verified results

Table 1 — Summary of verified outcomes

Area	Outcome	Status
FIM configuration	Downloads path added with `realtime` and `whodata`; agent log shows the path monitored (6003) and both engines started (6012, 6019). Service reported Running.	Verified
Full lifecycle detection	`wazuh_test2.txt`: added (rule 554, level 5), modified (rule 550, level 7), deleted (rule 553, level 7), each within about one second of the action.	Verified
Attribution	Every alert named user `Administrator` and process `powershell.exe`, PID 6324, mode `whodata`.	Verified
Hash evidence	Before and after hashes captured; MD5 chain across add, modify and delete is continuous. The deletion alert carries the last-known hashes.	Verified
Deleted-file rule ID	Rule 553 captured on screen. This closes the evidence gap recorded in Day 9.	Verified

Area	Outcome	Status
Fast-change limitation	Edit plus delete in the same second produced only added and deleted alerts for `wazuh_test.txt`; no modified alert exists.	Observed; cause not proven
Real browser download	Not tested. All activity came from PowerShell.	Untested

> **Scope note.** This report covers the Wazuh FIM work only. It is a continuation of the Day 9 report and does not repeat the `C:\FIM_Test` exercises or any second-tool correlation.

2. Introduction and Objectives

2.1 Background and motivation

Day 9 used a dedicated folder so that every alert could be traced to a deliberate action. That was the right way to learn the tool, but real endpoints rarely behave so tidily. Files arrive in Downloads from browsers, email clients and chat applications; they are opened, edited, renamed and deleted by users. Monitoring such a folder raises new questions about coverage, noise and attribution. PCI DSS requirement 11.5 and NIST SP 800-53 control SI-7 (both tagged on the alerts seen here) expect organisations to detect unauthorised changes to important files, and a landing folder for external content is an obvious place to apply that thinking.

2.2 Objectives

1. Add the Windows Downloads folder to Wazuh FIM safely and reversibly.
2. Confirm from the agent log that the folder is monitored in real time with who-data.
3. Generate file creation, modification and deletion events in that folder.
4. Record path, user, process, time, size, hashes and rule identity for each event.
5. Identify the limits of the approach, including noise, fast changes and what remains untested.

2.3 Scope

In scope: one Windows 10 endpoint (agent ID 006), one additional monitored folder, and the Wazuh manager and dashboard on the Ubuntu VM. **Out of scope:** custom rules, active response, alert forwarding, malware samples, and any monitoring of other user profiles or system directories. Nothing outside the isolated VirtualBox host-only network was touched.

2.4 Measurable success criteria

Table 2 — Success criteria and result

ID	Criterion	Result
S1	The Downloads path appears in the agent log as monitored, with real-time and who-data engines started.	Met (Figure 5)
S2	A file created in Downloads produces an alert naming path, user, process and hashes.	Met (Figure 7, Figure 10, Figure 11)
S3	A modification produces an alert with before and after hashes, sizes and the same attribution.	Met (Figure 17 to Figure 20)
S4	A deletion produces an alert with user, process, last-known hashes and a captured rule ID.	Met (rule 553; Figure 21 to Figure 24)
S5	The hash chain across the lifecycle is continuous.	Met for MD5; SHA-256 partly captured
S6	Limits of the approach are identified from lab observations.	Met (Chapters 11 and 16)
S7	A file saved by a real browser download is detected and attributed.	**Not tested**

3. Technical Background

Each idea below is explained in plain language first and then technically.

3.1 File Integrity Monitoring (FIM)

Plain language: FIM is a security camera for files. It remembers what a file looked like and raises an alert when the picture changes. **Technical:** the Wazuh agent module `syscheck` records attributes of monitored files (size, permissions, owner, group, modification time, hashes). When a later scan or a real-time notification differs from the stored baseline, it sends a FIM event to the manager.

3.2 Hashes

A hash (MD5, SHA-1, SHA-256) is a fixed-length fingerprint calculated from a file's contents. Changing one character changes the fingerprint completely, and two files with identical content share the same fingerprint. The latter appears in this lab: `wazuh_test.txt` and `wazuh_test2.txt` were created with the same text and carry identical hashes, even though they are different files.

3.3 Scheduled, real-time and who-data modes

Table 3 — FIM monitoring modes

Mode	How it works	Seen in this lab
Scheduled	A full scan at an interval; the configured frequency is 43200 seconds (12 hours).	Registry alerts from 10:29 to 10:30 appear in the same list.
Real-time `realtime="yes"`	The operating system notifies the agent as soon as a change occurs.	Alerts within about one second of each action.
Who-data `whodata="yes"`	Uses Windows auditing (a system access control list, SACL, on the folder) to attach the user and process that made the change.	`syscheck.audit.user.name`, `syscheck.audit.process.name`, process ID.
Report changes `report_changes="yes"`	Stores a copy of text files so a content difference can be shown.	**Not enabled** for Downloads, by design.

3.4 From event to dashboard: six separate stages

Beginners often treat "a log appeared" as one step. It is at least six. Knowing which stage failed is the key to troubleshooting.

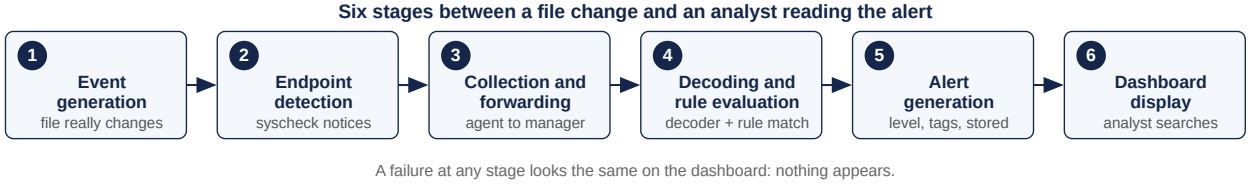


Figure 1 *— The six stages between a file change and an analyst reading the alert.*

Table 4 — Pipeline stages and where each was checked

Stage	Meaning	Where verified in this lab
1 Event generation	The file change really happens.	PowerShell commands (Figure 12).
2 Endpoint detection	syscheck notices it.	Agent log messages 6003, 6012, 6019 (Figure 5).
3 Collection and forwarding	The agent delivers the record.	Service Running (Figure 4); alerts reach the dashboard.
4 Decoding and rule evaluation	The manager splits fields and picks a rule.	`decoder.name`, `rule.id`.
5 Alert generation	A scored alert is stored.	`rule.level`, `rule.groups`, MITRE tags.
6 Dashboard display	The analyst finds it.	Integrity monitoring, Events view (Figure 7).

3.5 Rules and MITRE ATT&CK mapping

Wazuh ships rules for FIM events. Three matter here: rule 554 (file added), rule 550 (integrity checksum changed) and rule 553 (file deleted). Each carries a severity level and compliance tags (PCI DSS 11.5, NIST 800-53 SI.7, GDPR II_5.1.f, HIPAA 164.312). The modification and deletion rules also carry MITRE ATT&CK tags, which are presented in Chapter 7.

4. Lab Environment and Architecture

4.1 Components

Table 5 — Laboratory components (as evidenced by the screenshots)

Component	Details
Virtualisation	Oracle VirtualBox; a Windows 10 guest and an Ubuntu guest (`Ubuntu_SnapShot`).
Monitored endpoint	Windows 10 host `DESKTOP-T6T1N2V`, IP `192.168.56.109`, Wazuh agent ID `006`, service `WazuhSvc` (display name "wazuh"). Operating system build 10.0.19045.6466 as documented in Day 9.
Wazuh server	Manager name "Ubuntu"; dashboard reached at `192.168.56.101`. The Wazuh version was not captured in the evidence.
Alert index	`wazuh-alerts-4.x-2026.11.12`.
Account used	Local `Administrator` (SID ending `-500`) in an elevated PowerShell session.
Network	The VirtualBox host-only range `192.168.56.0/24`; no external network is needed for the tests.

4.2 Architecture and data flow

The agent on the Windows VM watches two folders. It asks Windows to audit the Downloads folder so that the user and process can be recorded, and it forwards each FIM event to the manager on the Ubuntu VM. There the event is decoded, matched to a rule, stored as an alert and made searchable in the dashboard.

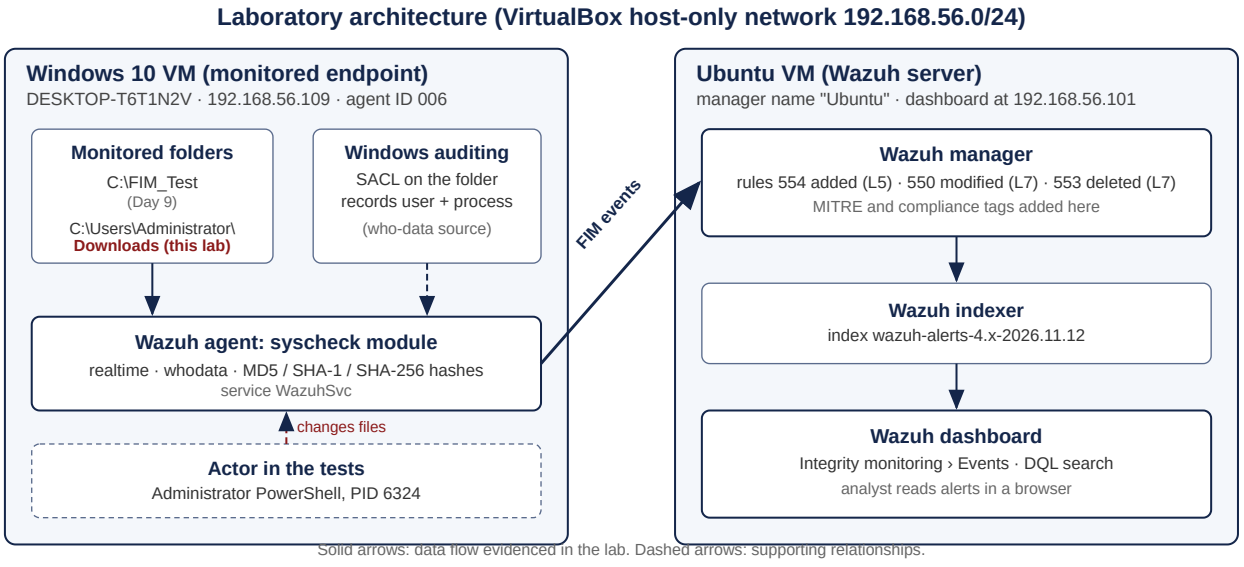


Figure 2 *— Laboratory architecture and data flow. Drawn from the environment evidenced in the screenshots.*

5. Prerequisites and Safety

5.1 Prerequisites

- A working Wazuh manager, indexer and dashboard, with the Windows 10 machine enrolled and showing as active.
- Administrator rights on the Windows VM: needed to edit the agent configuration, restart the service and let who-data place audit entries.
- A browser on the Ubuntu VM for the Wazuh dashboard.
- The Day 9 configuration already in place (the `C:\FIM_Test` line); this lab adds one line beside it.

5.2 Safety measures used

- **Backup before change:** `ossec.conf` was copied to `ossec.conf.bak2` (the Day 9 backup, `ossec.conf.bak`, was left untouched).
- **One change only:** a single configuration line was added.
- **Harmless content:** the test files contain dummy text.
- **Privacy-aware option choice:** `report_changes` was left off for Downloads because it would copy the content of every text file into the agent's diff store.
- **Isolation:** only the isolated host-only network was used.

> **Warning.** Do not enable real-time and who-data monitoring on a busy production Downloads folder without testing. Browsers create and rename temporary files, and each step can raise alerts.

5.3 Restoring the lab if something goes wrong

Documented for the reader; the restore was not needed in this lab.

```
Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf.bak2" "C:\Program Files (x86)\ossec-agent\ossec.conf" -Force
Restart-Service -Name wazuh
```

6. Step-by-Step Implementation

All times are the Windows VM clock as printed by `Get-Date` or shown in the taskbar, and the Wazuh alert timestamp as displayed by the dashboard. The two agree to within about one second for every event that was timed (see Section 16.2 for a display anomaly in one field). "PS>" means an Administrator PowerShell prompt on the Windows VM.

6.1 Step 1 — Find the exact Downloads path

What / where	Print the current user's Downloads path. Windows VM, Administrator PowerShell.
Why	The agent needs the literal path. Using the environment variable avoids typing the wrong profile name.
Command	`echo $env:USERPROFILE\Downloads`
Meaning	`$env:USERPROFILE` expands to the signed-in user's profile folder; `\Downloads` is appended.
Expected	A path of the form `C:\Users\<name>\Downloads`.
Actual	`C:\Users\Administrator\Downloads` (first output line of Figure 3).
Common errors	Browser set to save elsewhere: check the browser's download location before assuming this path is the right one.

6.2 Step 2 — Back up and edit the agent configuration

What / where	Copy `ossec.conf`, open it in Notepad, add one line. Windows VM, Administrator PowerShell.
Why	The agent monitors only what its configuration lists. A backup allows a clean rollback.
Commands	`Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" "C:\Program Files (x86)\ossec-agent\ossec.conf.bak2"` `notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"`
Expected	A new file `ossec.conf.bak2`; Notepad opens the live configuration.
Actual	Both commands ran without error. Notepad's title bar shows `*ossec.conf` (an asterisk marks unsaved edits) and the new line sits directly under the Day 9 line (Figure 3).

The line added under `<frequency>43200</frequency>`, beside the existing `C:\FIM_Test` entry, was:

```
<directories check_all="yes" realtime="yes"
whodata="yes">C:\Users\Administrator\Downloads</directories>
```

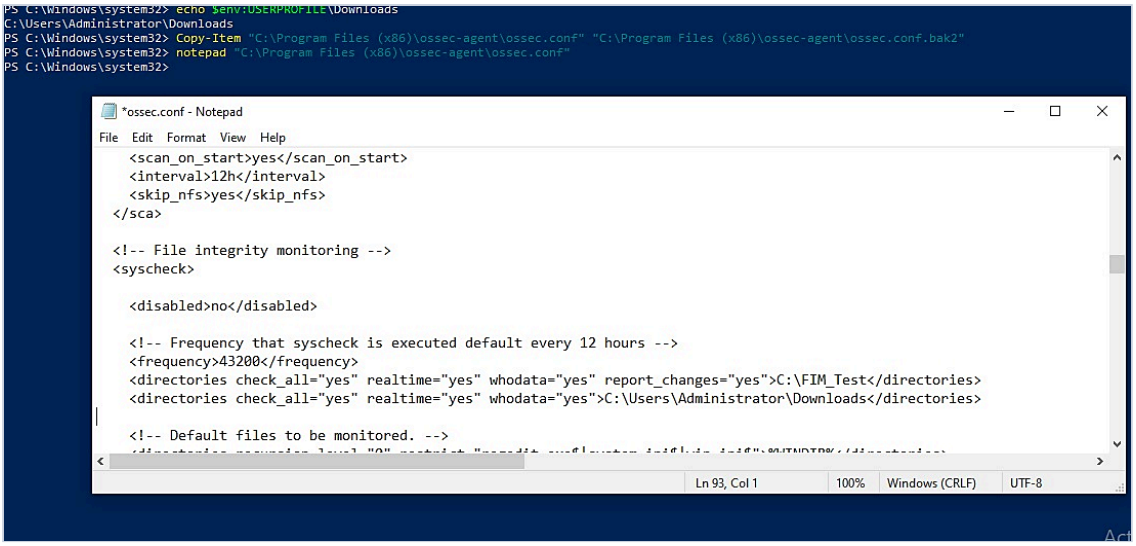


Figure 3 *— Backup command, Notepad open on ossec.conf, and the new Downloads line beneath the Day 9 FIM_Test line. Cropped to the relevant window.*

> **Note.** `report_changes="yes"` was intentionally omitted. For Downloads the aim is detection and attribution, not storing copies of whatever was downloaded.

6.3 Step 3 — Restart the agent and check the service

Commands	`restart-service -name wazuh` `get-service -name wazuh`
Why	The agent reads `ossec.conf` only at start-up.
Expected	Status `Running`.
Actual	Service `WazuhSvc` (display name "wazuh") reported `Running` (Figure 4).
Common errors	Status Stopped usually means malformed XML. Restore the backup (Section 5.3) and repeat.



Figure 4 *— The same session after the edit: service restart and Running status.*

6.4 Step 4 — Verify that FIM started (agent log)

Command	`Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.log" -Pattern "downloads|whodata|real-time" | Select-Object -Last 10`
Meaning	`Select-String` searches the log for any of three words (the `|` inside the quotes means "or"); `Select-Object -Last 10` keeps the final ten matches.
Expected	The path reported as monitored, then the real-time and who-data engines started.
Actual	Five lines, transcribed below (Figure 5).
Why it matters	This is pipeline stage 2. Without these lines, later "no results" could not be interpreted.

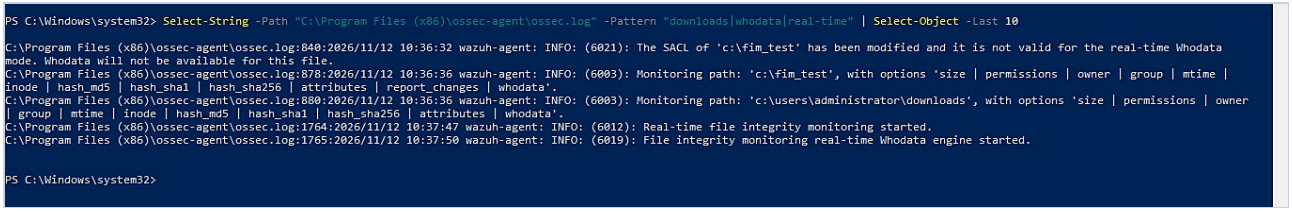


Figure 5 *— Agent log output confirming the Downloads path, real-time monitoring and the who-data engine. Transcription follows.*

Table 6 — Transcription of the log lines shown in the figure above

Time	ID	Message (abridged only where stated)
10:36:32	6021	The SACL of `c:\fim_test` has been modified and it is not valid for the real-time Whodata mode. Whodata will not be available for this file.
10:36:36	6003	Monitoring path `c:\fim_test`, options: size, permissions, owner, group, mtime, inode, hash_md5, hash_sha1, hash_sha256, attributes, report_changes, whodata.
10:36:36	6003	Monitoring path `c:\users\administrator\downloads`, options: size, permissions, owner, group, mtime, inode, hash_md5, hash_sha1, hash_sha256, attributes, whodata. *(no report_changes)*
10:37:47	6012	Real-time file integrity monitoring started.
10:37:50	6019	File integrity monitoring real-time Whodata engine started.

> **Unresolved warning (6021).** Wazuh reports that who-data may not be available for `c:\fim_test`. The Downloads path produced no such warning and its alerts carry who-data fields. The likely cause is the agent re-applying audit rules on restart, but this was not confirmed, and `FIM_Test` was not retested afterwards.

6.5 Step 5 — First dashboard search: "No results"

Where	Wazuh dashboard on the Ubuntu VM: Integrity monitoring, Events tab, agent `DESKTOP-T6T1N2V (006)`.
Search	`syscheck.path: *downloads*` with the time range "a day ago to now".
Expected	Any events under Downloads.
Actual	"No results match your search criteria" at about 10:46 (Figure 6).
Interpretation	The first alert from Downloads arrives at 10:54:48 (Step 6). The empty result is therefore consistent with no file change having reached Wazuh before that. See Chapter 10.

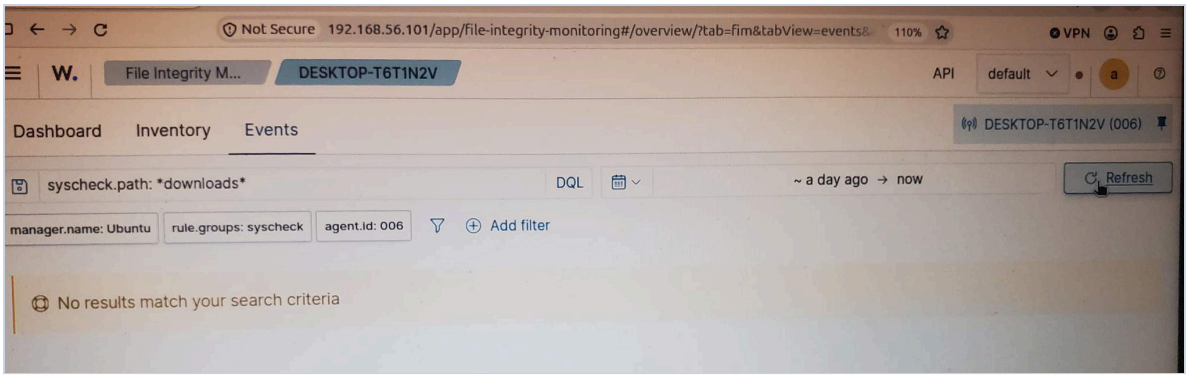


Figure 6 *— Dashboard Events search for syscheck.path: *downloads* returning no results. Photograph of the screen, straightened and cropped; browser tabs excluded. Screen clock reads Nov 12 10:46.*

6.6 Step 6 — Test 1: create a file

Command	`Get-Date; "download test" | Out-File C:\Users\Administrator\Downloads\wazuh_test.txt`
Meaning	`Get-Date` prints the time (an evidence anchor); the quoted text is piped into `Out-File`, which writes it to the file.
Expected	A "File added to the system" alert within seconds, with who-data.
Actual	Alert timestamped 10:54:48 (Figure 7). The screenshot of the command itself was not captured; the alert is the evidence that the file was created.

timestamp	agent.name	syscheck.path	syscheck.event	rule.des...
Nov 12, 2026 @ 10:54:48.2...	DESKTOP-T6T1N2V	c:\users\administrator\downloads\wazuh_test.txt	added	File added .
Nov 12, 2026 @ 10:36:32.4...	DESKTOP-T6T1N2V	-	-	Syscheck ..
Nov 12, 2026 @ 10:30:11.1...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	deleted	Registry Va
Nov 12, 2026 @ 10:30:10.5...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	deleted	Registry Va
Nov 12, 2026 @ 10:30:10.5...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	deleted	Registry Va
Nov 12, 2026 @ 10:30:10.5...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	deleted	Registry Va
Nov 12, 2026 @ 10:30:10.5...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	deleted	Registry Va
Nov 12, 2026 @ 10:29:53.4...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	modified	Registry Va
Nov 12, 2026 @ 10:29:53.4...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	modified	Registry Va
Nov 12, 2026 @ 10:29:53.4...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlS	modified	Registry Va
Nov 12, 2026 @ 10:29:53.4...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	modified	Registry Ke
Nov 12, 2026 @ 10:29:53.4...	DESKTOP-T6T1N2V	HKEY_LOCAL_MACHINE\System\CurrentControlSet\Ser...	modified	Registry Va

Figure 7 *— Events list: c:\users\administrator\downloads\wazuh_test.txt, event added, at the top; older registry alerts below.*

Expanding the alert shows the decoder and full log (Figure 8), the rule description and groups (Figure 9), who-data attribution (Figure 10) and the path, hashes and size (Figure 11).

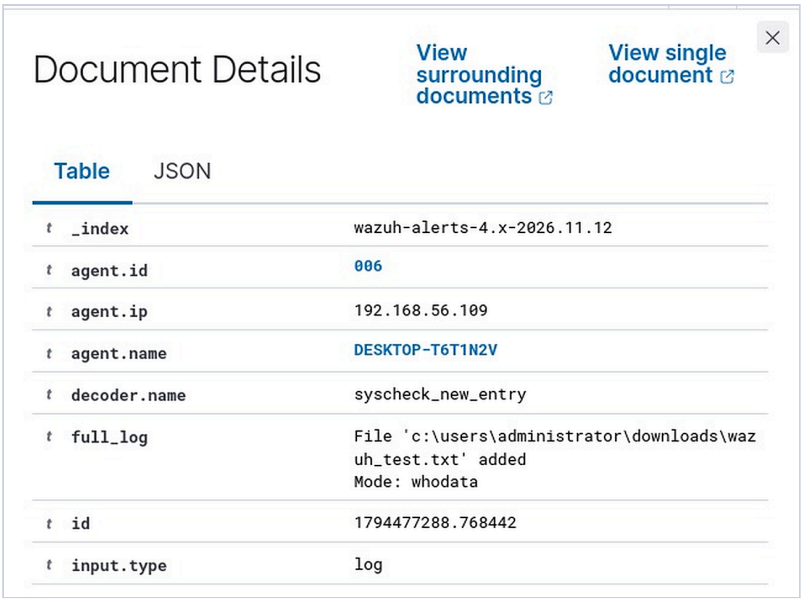


Figure 8 *— Creation alert: agent 006, DESKTOP-T6T1N2V, decoder syscheck_new_entry, full_log "File ... wazuh_test.txt' added, Mode: whodata".*

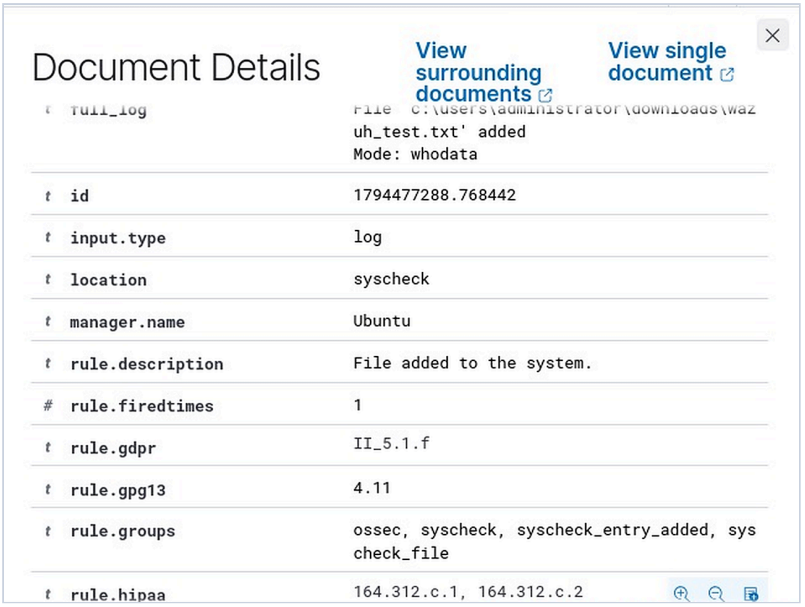


Figure 9 *— Creation alert: rule description "File added to the system.", firedtimes 1, groups ossec, syscheck, syscheck_entry_added, syscheck_file.*

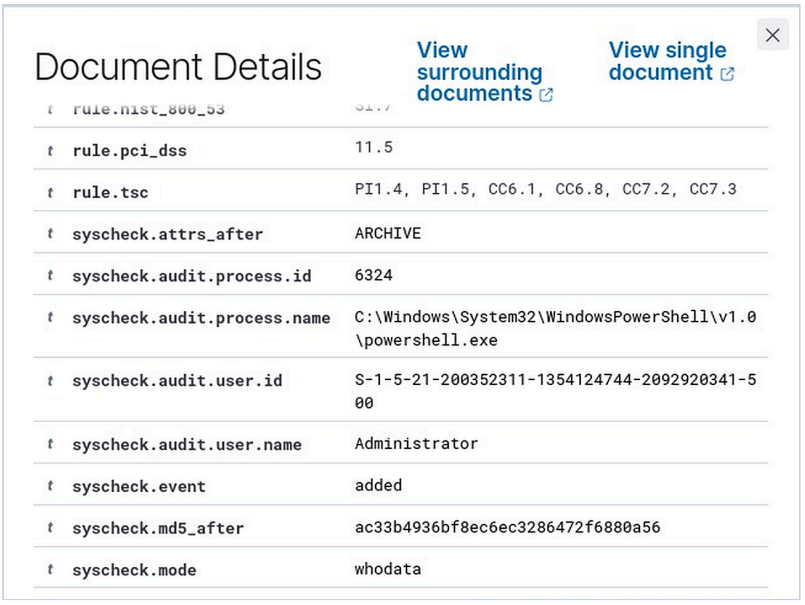


Figure 10 *— Creation alert: process powershell.exe PID 6324, user Administrator (SID ending -500), event added, mode whodata, MD5 ac33b4936bf8ec6ec3286472f6880a56.*

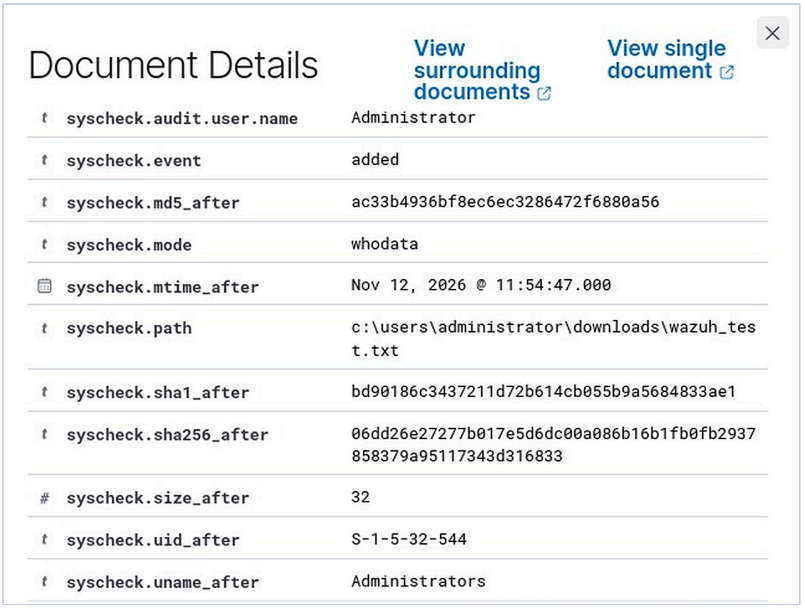


Figure 11 *— Creation alert: path, SHA-1 bd90186c..., SHA-256 06dd26e2..., size 32 bytes, owner group Administrators.*

Table 7 — Evidence captured for the creation of wazuh_test.txt

Field	Value
Alert time	10:54:48 (dashboard: 10:54:48.292)
Rule	"File added to the system." (group `syscheck_entry_added`); the numeric ID 554 is confirmed on the second test file (Figure 15)
Path	`c:\users\administrator\downloads\wazuh_test.txt`
User / process	Administrator / `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, PID 6324
Size	32 bytes

Field	Value
MD5	`ac33b4936bf8ec6ec3286472f6880a56`
SHA-1	`bd90186c3437211d72b614cb055b9a5684833ae1`
SHA-256	`06dd26e27277b017e5d6dc00a086b16b1fb0fb2937858379a95117343d316833`
Alert ID	`1794477288.768442`

Reader's note on size (inference, not shown on screen): 32 bytes for the 13-character line "download test" fits Windows PowerShell's default UTF-16 output (two bytes per character, a two-byte byte-order mark and a four-byte line ending: 26 + 2 + 4 = 32).

6.7 Step 7 — Test 2: edit and delete in the same second

The edit and the delete were pasted together and ran back to back (Figure 12).

Commands	`Get-Date; Add-Content C:\Users\Administrator\Downloads\wazuh_test.txt "My Name is Hope"` `Get-Date; Remove-Item C:\Users\Administrator\Downloads\wazuh_test.txt`
Meaning	`Add-Content` appends text to a file; `Remove-Item` deletes it.
Expected	A modified alert, then a deleted alert.
Actual	Both `Get-Date` outputs read Thursday, 12 November 2026, 11:19:33 AM. The events list shows a **deleted** alert at 11:19:34 and **no modified alert** for this file between the earlier added alert (10:54:48) and the delete (Figure 13).

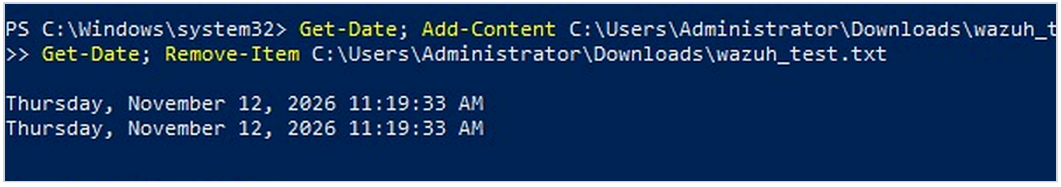


Figure 12 *— The edit and delete commands with their identical Get-Date outputs (11:19:33 AM). Enlarged crop of the PowerShell window; no error is displayed.*

Interpretation: with the two actions in one second, real-time FIM recorded only the final state change. Why exactly was not diagnosed. The screenshot shows no error from `Add-Content`, but it cannot prove the append completed before the delete. The lesson is practical: **a missing "modified" alert does not prove nothing was modified.** Step 8 was therefore repeated with pauses.

6.8 Step 8 — Test 3: a clean add, modify, delete lifecycle

Commands	Run one at a time, waiting about ten seconds between them:
	`Get-Date; "download test" | Out-File C:\Users\Administrator\Downloads\wazuh_test2.txt`
	`Get-Date; Add-Content C:\Users\Administrator\Downloads\wazuh_test2.txt "My Name is Hope"`
	`Get-Date; Remove-Item C:\Users\Administrator\Downloads\wazuh_test2.txt`

Expected	Three alerts in order: added (554), modified (550), deleted (553).
Actual	Exactly that, at 11:27:10, 11:28:20 and 11:30:20 (Figure 13). The `Get-Date` screenshots for this run were not captured, so action times are inferred from the alerts.

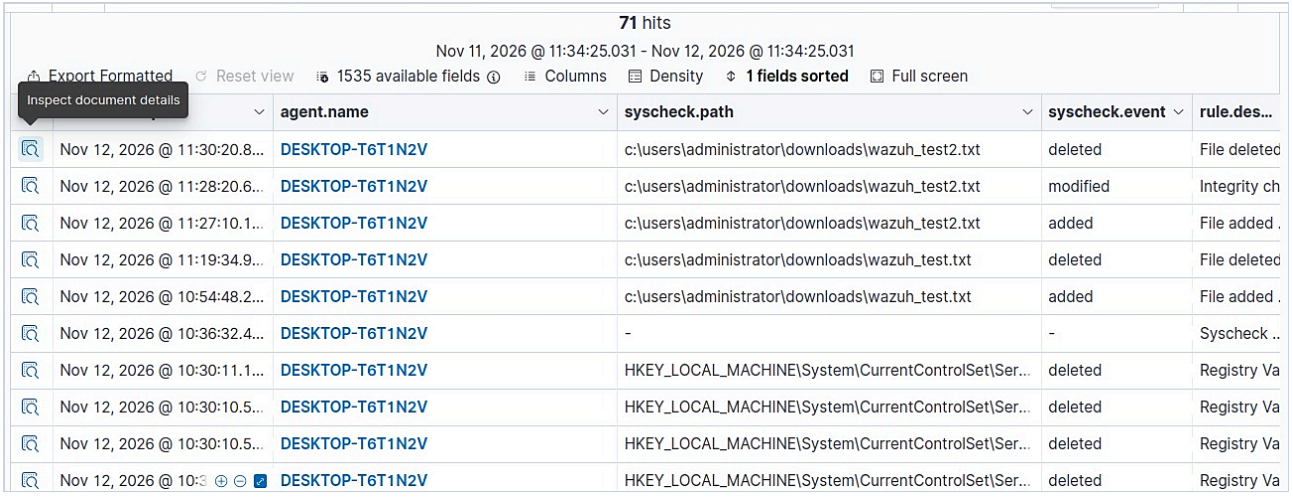


Figure 13 *— Events list showing the full set of file events: wazuh_test2.txt deleted, modified, added; wazuh_test.txt deleted and added; the 10:36:32 warning row; registry noise below. 71 hits.*

8a. The creation alert

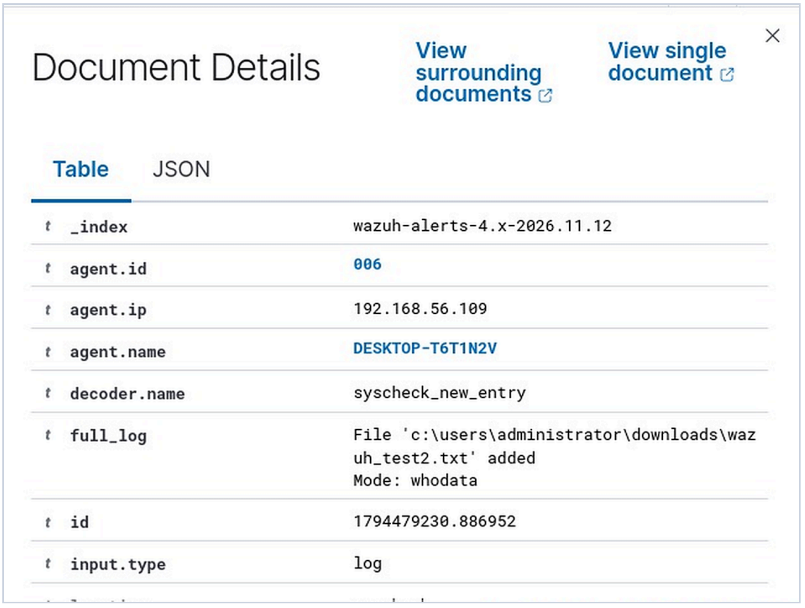


Figure 14 *— wazuh_test2.txt creation: decoder syscheck_new_entry, full_log "added, Mode: whodata", alert ID 1794479230.886952.*

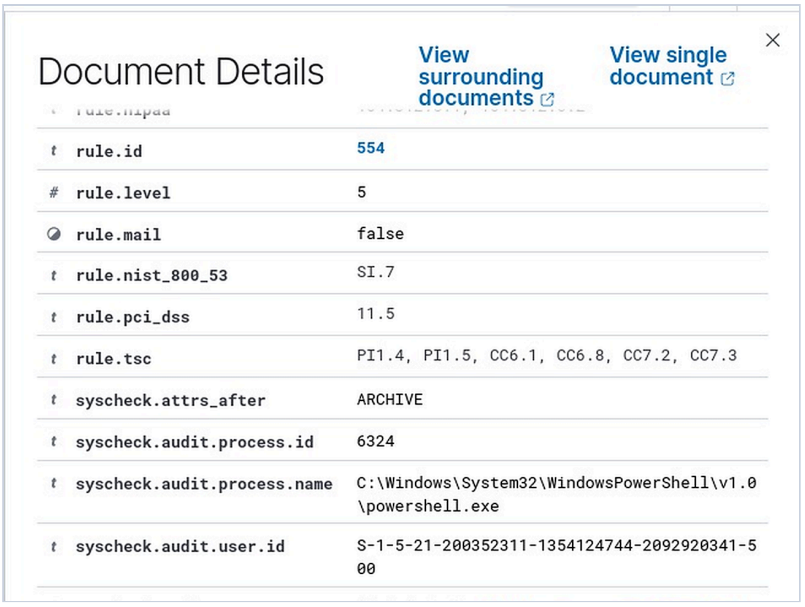


Figure 15 *— Creation alert: rule 554, level 5, PCI DSS 11.5, NIST SI.7; process powershell.exe PID 6324, user SID ending -500.*

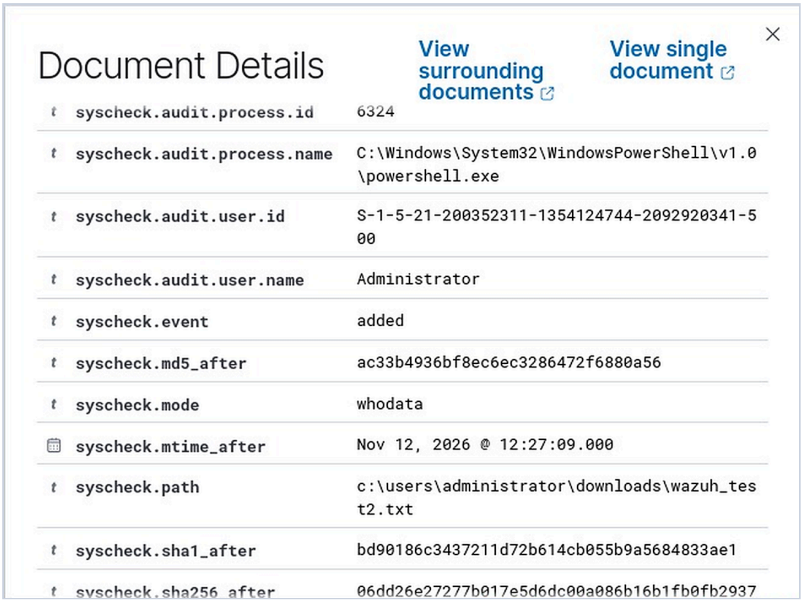


Figure 16 *— Creation alert: user Administrator, event added, MD5 ac33b4936bf8ec6ec3286472f6880a56, mode whodata, SHA-1 bd90186c..., path wazuh_test2.txt.*

8b. The modification alert

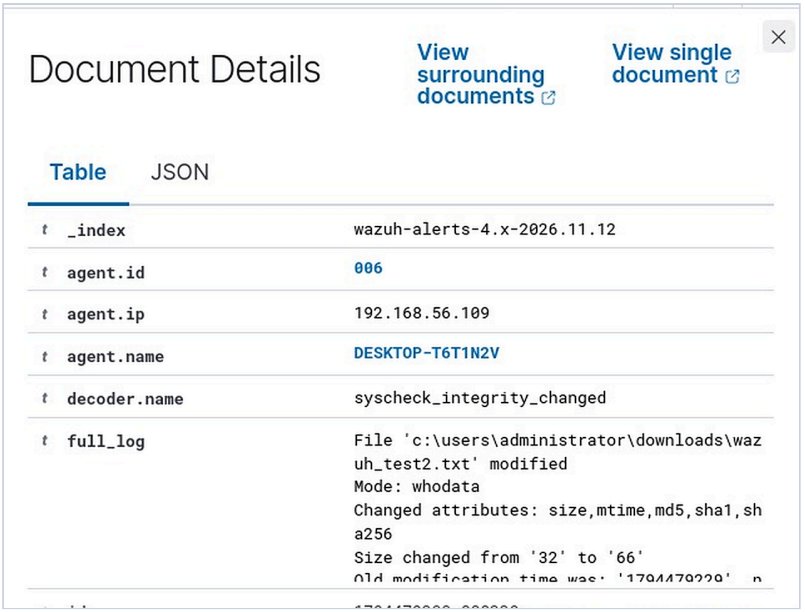


Figure 17 *— Modification alert: decoder syscheck_integrity_changed; changed attributes size, mtime, md5, sha1, sha256; "Size changed from '32' to '66'".*

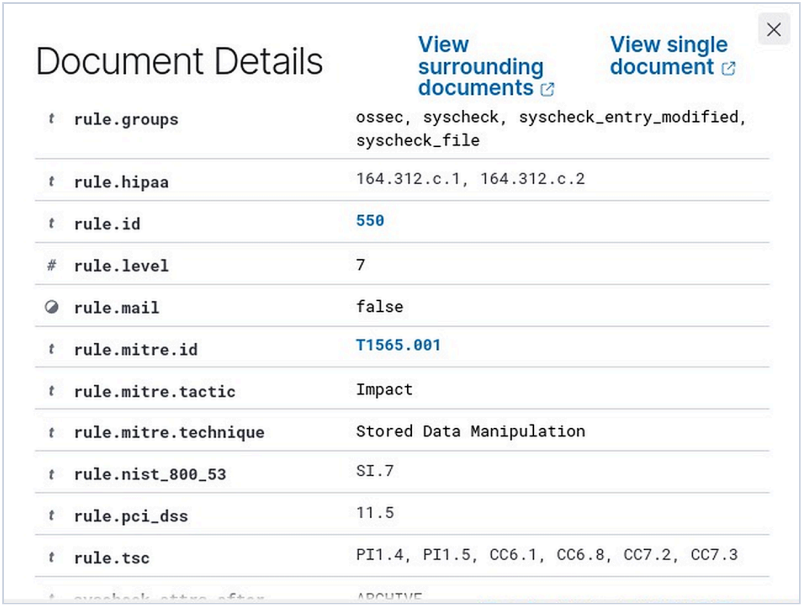


Figure 18 *— Modification alert: rule 550, level 7, groups include syscheck_entry_modified, MITRE T1565.001 (Stored Data Manipulation, tactic Impact).*

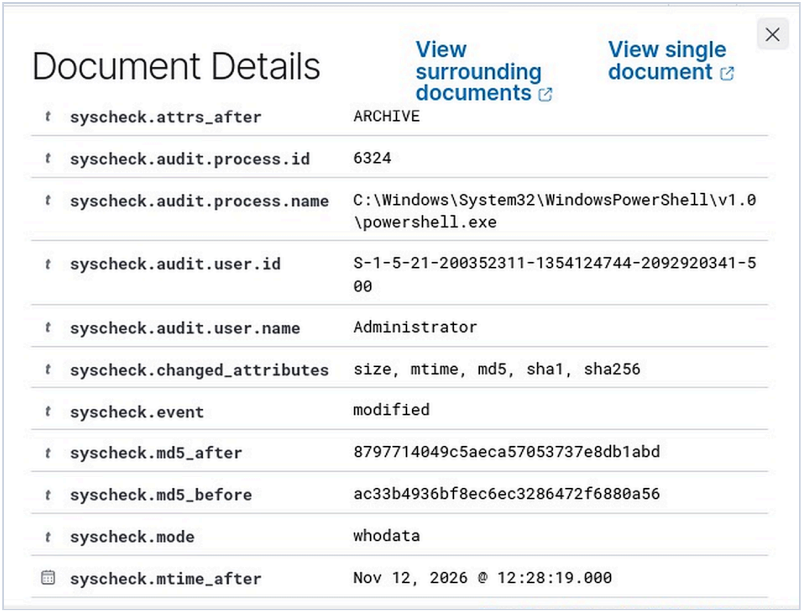


Figure 19 *— Modification alert: same user and process (PID 6324); MD5 before ac33b4936bf8ec6ec3286472f6880a56, after 8797714049c5aeca57053737e8db1abd.*

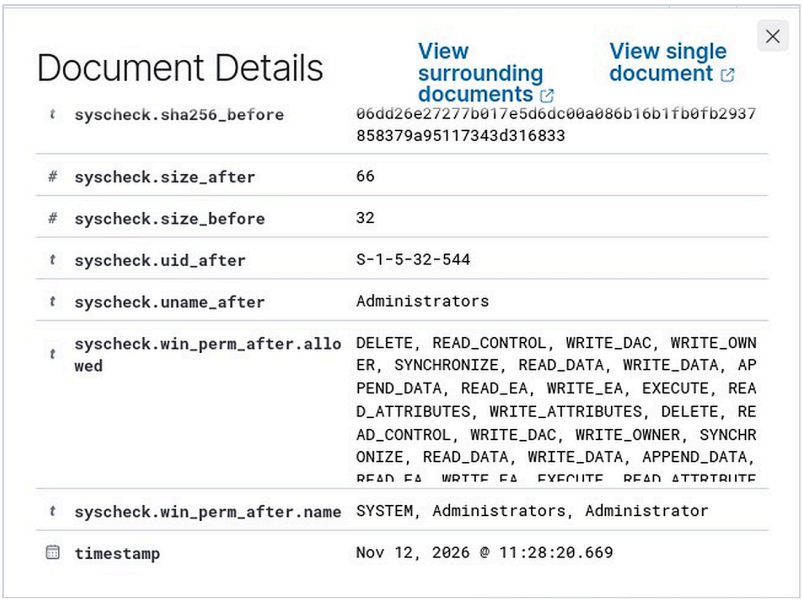


Figure 20 *— Modification alert: SHA-256 before 06dd26e2..., size 32 to 66, alert timestamp 11:28:20.669.*

Table 8 — Evidence captured for the modification of wazuh_test2.txt

Field	Before	After
Size	32	66
MD5	`ac33b4936bf8ec6ec3286472f6880a56`	`8797714049c5aeca57053737e8db1abd`
SHA-256	`06dd26e27277b017e5d6dc00a086b16b1fb0fb2937858379a95117343d316833`	not captured on this alert (see the deletion alert)
Rule	550, level 7, MITRE T1565.001; alert at 11:28:20.669	
User / process	Administrator / `powershell.exe`, PID 6324, mode whodata	

The 34-byte growth fits a 15-character line in UTF-16 plus a four-byte line ending (30 + 4); an inference, not displayed on screen.

8c. The deletion alert

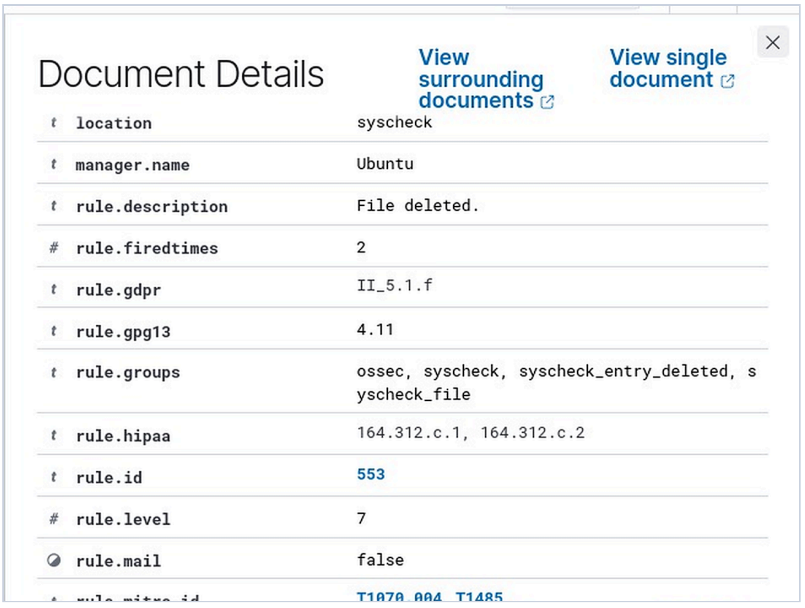


Figure 21 *— Deletion alert: rule 553, level 7, description "File deleted.", groups include syscheck_entry_deleted, firedtimes 2.*

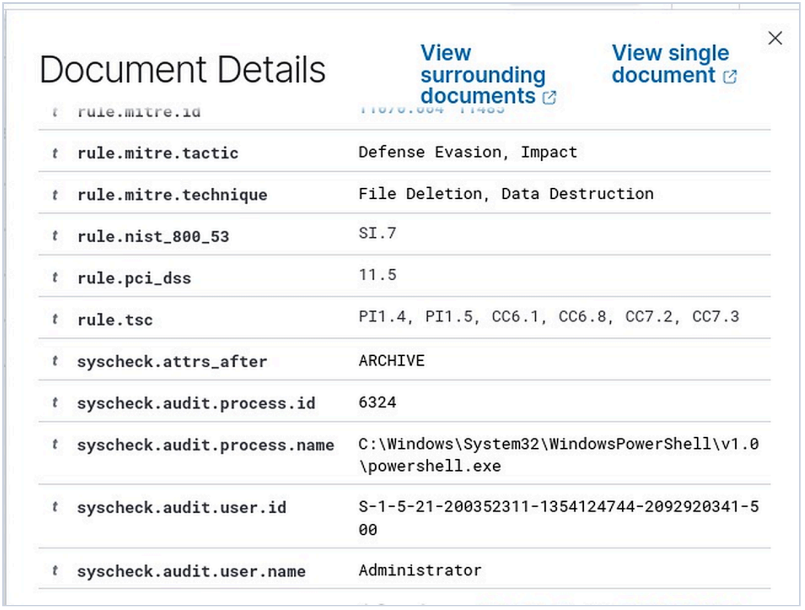
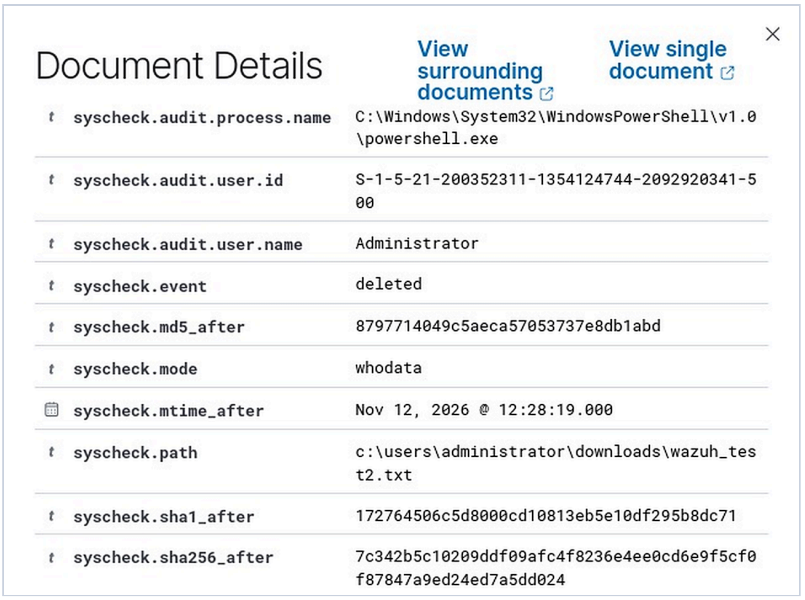


Figure 22 *— Deletion alert: MITRE tactics Defense Evasion and Impact; techniques File Deletion and Data Destruction; process PowerShell PID 6324.*

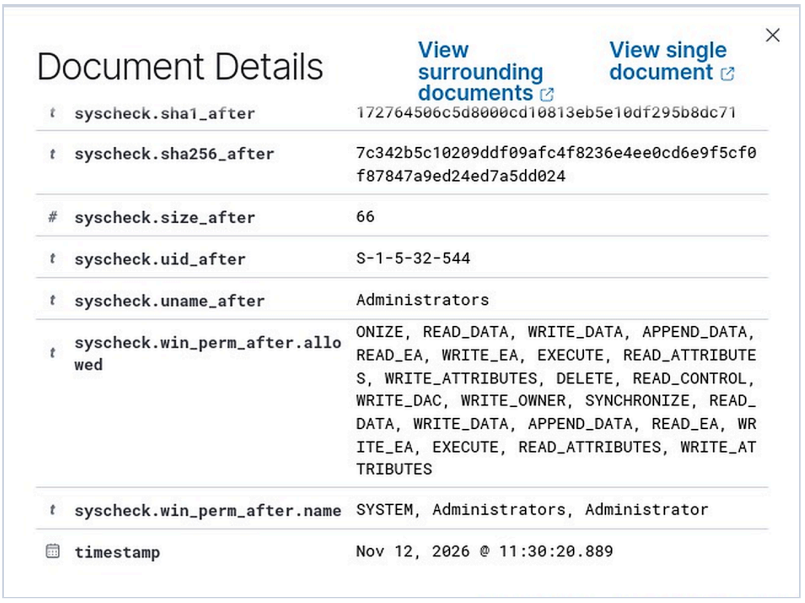


Document Details

View surrounding documents | View single document

t syscheck.audit.process.name	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
t syscheck.audit.user.id	S-1-5-21-200352311-1354124744-2092920341-500
t syscheck.audit.user.name	Administrator
t syscheck.event	deleted
t syscheck.md5_after	8797714049c5aeca57053737e8db1abd
t syscheck.mode	whodata
syscheck.mtime_after	Nov 12, 2026 @ 12:28:19.000
t syscheck.path	c:\users\administrator\downloads\wazuh_test2.txt
t syscheck.sha1_after	172764506c5d8000cd10813eb5e10df295b8dc71
t syscheck.sha256_after	7c342b5c10209ddf09afc4f8236e4ee0cd6e9f5cf0f87847a9ed24ed7a5dd024

Figure 23 *— Deletion alert: event deleted, last-known MD5 8797714049c5aeca57053737e8db1abd, SHA-1 172764506c5d8000cd10813eb5e10df295b8dc71, SHA-256 7c342b5c....*



Document Details

View surrounding documents | View single document

t syscheck.sha1_after	172764506c5d8000cd10813eb5e10df295b8dc71
t syscheck.sha256_after	7c342b5c10209ddf09afc4f8236e4ee0cd6e9f5cf0f87847a9ed24ed7a5dd024
# syscheck.size_after	66
t syscheck.uid_after	S-1-5-32-544
t syscheck.uname_after	Administrators
t syscheck.win_perm_after.allowed	ONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES
t syscheck.win_perm_after.name	SYSTEM, Administrators, Administrator
timestamp	Nov 12, 2026 @ 11:30:20.889

Figure 24 *— Deletion alert: last-known SHA-256 7c342b5c10209ddf09afc4f8236e4ee0cd6e9f5cf0f87847a9ed24ed7a5dd024, size 66, alert timestamp 11:30:20.889.*

Evidence. The deleted file's last-known MD5 equals the modification alert's "after" MD5, and the modification's "before" SHA-256 equals the creation alert's SHA-256. The file was therefore deleted in exactly the state in which it was last edited. The modification alert's own "after" SHA-256 is not on a captured screenshot, so the SHA-256 chain is shown at its two ends rather than every link.

7. Configuration and Detection Logic

7.1 The configuration change (the only one made)

```
<directories check_all="yes" realtime="yes"
whodata="yes">C:\Users\Administrator\Downloads</directories>
```

Table 9 — Line-by-line explanation

Part	Meaning
`<directories ...>`	Tells syscheck to monitor the listed path.
`check_all="yes"`	Record size, permissions, owner, group, modification time, inode, MD5, SHA-1 and SHA-256 (the 6003 message lists these options).
`realtime="yes"`	Report changes as they happen rather than at the next scheduled scan.
`whodata="yes"`	Use Windows auditing to attach user and process. Implies real-time.
(no `report_changes`)	No stored copy of file contents and no line-by-line difference. Chosen on purpose.
`C:\Users\Administrator\Downloads`	The monitored directory. Wazuh lower-cases it in alerts.

7.2 Original, changed and recommended

Table 10 — Configuration status

Setting	State	Evidence
`<frequency>43200</frequency>` (12 h)	Original	Figure 3
Day 9 `C:\FIM_Test` line with `report_changes`	Original to this lab	Figure 3
Downloads line (above)	**Changed**	Figure 3, Figure 5
Ignore rules for browser temporary files	Recommended only	Chapter 12
Custom rule for executables and scripts in Downloads	Recommended only	Chapter 12

7.3 Rules that fired

Table 11 — FIM rules observed for the Downloads folder

Rule	Level	Description	Group and tags observed
554	5	File added to the system.	`syscheck_entry_added`; PCI DSS 11.5; NIST SI.7; GDPR II_5.1.f; HIPAA 164.312.c.1/c.2; GPG13 4.11
550	7	Integrity checksum changed.	`syscheck_entry_modified`; MITRE T1565.001 (Stored Data Manipulation), tactic Impact

Rule	Level	Description	Group and tags observed
553	7	File deleted.	`syscheck_entry_deleted`; MITRE T1070.004 and T1485 (File Deletion; Data Destruction), tactics Defense Evasion and Impact

Rule 553 reported `firedtimes: 2`, consistent with the two deletions made that day.

7.4 Fields used for detection and investigation

Table 12 — Wazuh FIM fields used in this lab

Field	Use
`rule.id`, `rule.level`, `rule.description`, `rule.groups`	Which rule fired, how severe, in which family.
`syscheck.path`, `syscheck.event`	Which file and what happened (added, modified, deleted).
`syscheck.mode`	`whodata` (real-time with attribution) or `scheduled`.
`syscheck.audit.user.name`, `.user.id`	Who made the change; SID ending `-500` is the built-in Administrator.
`syscheck.audit.process.name`, `.process.id`	Which program and which instance.
`syscheck.md5_*`, `sha1_*`, `sha256_*`, `size_*`	Content fingerprint and size, before and after.
`syscheck.changed_attributes`	Which attributes differ (here size, mtime, md5, sha1, sha256).
`full_log`, `decoder.name`	Human-readable summary and which decoder parsed it.

7.5 Dashboard searches

The Events tab uses the dashboard query language (DQL). One search is visible in the evidence (Figure 6); the others were used to isolate the test files.

Table 13 — Searches used

Query	Purpose
`syscheck.path: *downloads*`	Every FIM event under any Downloads path. Visible in Figure 6.
`syscheck.path: *wazuh_test*`	Narrow to the test files (also matches `wazuh_test2.txt`). Query text not captured in a screenshot.

8. Testing and Results

8.1 Test scenarios

Table 14 — Test scenarios, expectations and outcomes

#	Scenario	Expected	Outcome	Status
T1	Create `wazuh_test.txt`	Rule 554 alert with who-data	Added at 10:54:48; PowerShell PID 6324.	Verified
T2	Edit and delete in the same second	Modified, then deleted	Deleted at 11:19:34; no modified alert.	Observed limitation
T3	Create, edit, delete `wazuh_test2.txt` with pauses	554, 550, 553	All three, in order, each with attribution.	Verified
T4	File saved by a web browser	Added alert naming the browser process	Not performed.	Untested
T5	FIM_Test who-data after restart	Who-data still present	Warning 6021 logged; not retested.	Unresolved

8.2 File timeline (required deliverable)

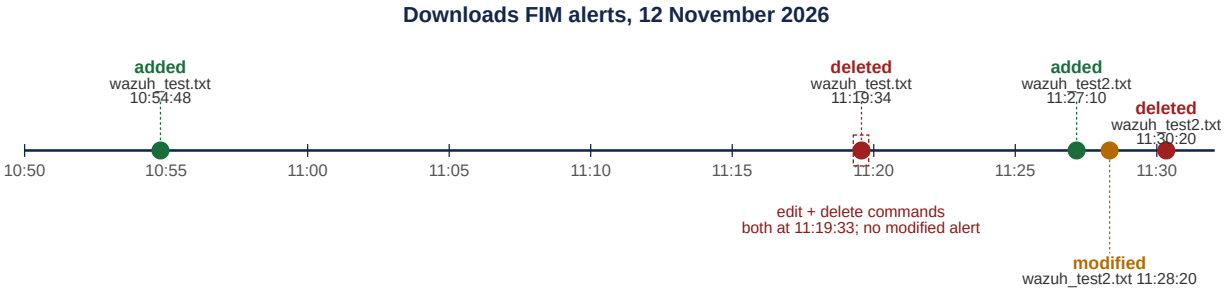


Figure 25 *— Timeline of Downloads FIM alerts on 12 November 2026 (alert timestamps, local time).*

Table 15 — File timeline

Time	File	Event	Rule	Notes
10:54:48	`wazuh_test.txt`	added	description only	32 bytes; SHA-256 06dd26e2...
11:19:34	`wazuh_test.txt`	deleted	not shown	Edit and delete commands both ran at 11:19:33; no modified row.
11:27:10	`wazuh_test2.txt`	added	554 (L5)	32 bytes; same hashes as `wazuh_test.txt`.
11:28:20	`wazuh_test2.txt`	modified	550 (L7)	32 to 66 bytes; MD5 ac33... to 8797...
11:30:20	`wazuh_test2.txt`	deleted	553 (L7)	Last-known MD5 8797...; SHA-256 7c342b5c...

8.3 What succeeded, failed and remains untested

- **Succeeded:** path monitored in real time with who-data; add, modify, delete alerts with user, process and hashes; rule 553 captured.
- **Did not behave as expected:** no modified alert for the edit-then-delete in one second.
- **Untested:** browser downloads (and the temporary files and renames they create), other applications such as Word or Notepad, files other than plain text, sub-folders, and large or numerous files.

9. Evidence and Screenshot Analysis

9.1 Evidence handling

Original screenshots are retained unchanged. Presentation copies were produced only by cropping out unrelated areas (the VirtualBox status strip, desktop and empty space), upscaling with a standard resampling filter and a light sharpen. One photograph of a screen was rotated by about 1.3 degrees to straighten it. Nothing was redrawn, retouched or annotated, and no value was altered. Several screenshots that repeated information already shown were omitted from the main text and are listed in the evidence register (Appendix C).

9.2 Background alerts seen alongside the test

The events list also contains alerts that are **not** part of the experiment (Figure 7, Figure 13):

- A cluster of `HKEY_LOCAL_MACHINE\System\CurrentControlSet\...` registry alerts (*modified* at 10:29:53, *deleted* at 10:30:10 to 10:30:11) in scheduled mode, before the configuration was edited. These are routine background noise.
- A row at 10:36:32 with no path or event, whose rule description begins "Syscheck". Its time matches log message 6021, so it is the who-data warning for `FIM_Test` surfacing as an alert.

9.3 What each evidence group proves

Table 16 — Evidence assessment

Evidence	Demonstrates	Limitation
Figure 3, Figure 4, Figure 5	The configuration line, a backup, a Running service and monitoring started for the Downloads path.	Log lines are a point-in-time search; line wraps are the console's.
Figure 7, Figure 13	Which alerts exist, in what order and when.	Table columns are truncated at the right edge.
Figure 8 to Figure 11	Creation alert contents for test 1.	Numeric rule ID not visible for this alert.
Figure 14 to Figure 24	Full lifecycle contents for test 2.	Modification alert's "after" SHA-256 not captured.
Figure 12	Both commands ran in the same second.	Cannot show that the append completed.

Evidence	Demonstrates	Limitation
Figure 6	The empty search result at 10:46.	A photograph; shows the state at one moment.

10. Troubleshooting and Root-Cause Analysis

Table 17 — Problems encountered

Symptom	Possible causes	Diagnosis and outcome
Dashboard shows "No results" for `syscheck.path : *downloads*` (10:46)	(a) Monitoring not active. (b) No file change had happened in the monitored folder. (c) A browser had saved the file elsewhere. (d) Search window or filters wrong.	Log shows monitoring active from 10:37 (rules out a). The time range was a full day and the filters were the standard three (rules out d). A controlled file created in the folder produced an alert at 10:54:48. **Most probable cause: (b) or (c); not confirmed**, because what, if anything, was downloaded before 10:46 is unknown.
Warning 6021 for `c:\fim_test`	Audit rule (SACL) on the folder changed or replaced when the agent restarted.	Not confirmed. Downloads is unaffected. Retest FIM_Test and look for 6019 and the audit fields.
No modified alert for `wazuh_test.txt`	Edit and delete in one second; events coalesced or the edit not registered separately.	Reproduced with pauses (Test 3): all three alerts appeared. Mechanism not diagnosed.
`mtime_after` one hour later than the alert time	Display or time-zone handling of the mtime field.	See Section 16.2.

11. Security Findings and Risk Analysis

11.1 Findings

Table 18 — Findings and confidence

ID	Finding	Evidence	Confidence
F1	Adding one configuration line is enough to monitor a user-writable folder in real time with attribution.	Figure 3, Figure 5, Figure 10	High
F2	Each lifecycle step produces a distinct, scored alert naming user, process and hashes.	Figure 13 to Figure 24	High

ID	Finding	Evidence	Confidence
F3	The MD5 chain is continuous, so the file's state can be reconstructed from alerts alone.	Figure 16, Figure 19, Figure 23	High
F4	Very fast changes can be collapsed: a missing alert does not prove no change.	Figure 12, Figure 13	Medium (cause unproven)
F5	An ordinary delete of a user's own file raises a level 7 alert tagged Defense Evasion and Data Destruction. In Downloads, where deleting files is normal, this is a false-positive risk.	Figure 21, Figure 22	High
F6	Who-data can be unavailable for a folder even while the agent runs.	6021 for FIM_Test (Figure 5)	Medium
F7	Attribution names the program that wrote the file (here, PowerShell), not the person's intent.	All alerts	High

11.2 Analyst notes: the questions an analyst asks

Question	Answer from this lab
What changed?	`wazuh_test2.txt` was added, then grew from 32 to 66 bytes, then was deleted.
Where?	`c:\users\administrator\downloads` on `DESKTOP-T6T1N2V`.
Who?	User `Administrator`, SID ending `-500`.
With what?	`powershell.exe`, PID 6324.
When?	11:27:10, 11:28:20 and 11:30:20 on 12 November 2026.
Why, and was it approved?	Not answerable from the alerts. Here the author knows it was a test; in real work this needs a ticket, the user, or further evidence.

11.3 Why context matters

"A file was deleted" is neutral. The same rule 553 alert could mean a user tidying a folder, or malware removing its dropper. What separates them is context: the file type, the program that did it, the time, the account, and whether it follows an unusual add. A level 7 score on every delete is a prompt to look, not a verdict.

12. Mitigation and Remediation

12.1 Implemented in the lab

Table 19 — Controls actually implemented

Control	Detail
Configuration backup	`ossec.conf.bak2` created before editing.
Minimal change	One line added; nothing else modified.
Data minimisation	`report_changes` omitted for Downloads.
Verification	Service status and agent log checked before testing.
Retest with pauses	Repeated the lifecycle with delays after the same-second case.

12.2 Recommendations (not implemented)

None of the following was applied in this lab. The code is illustrative and must be tested before use.

Table 20 — Recommended improvements

#	Recommendation	Objective and approach	Verify, impact, residual risk
R1	Ignore browser temporary files	Cut noise from partial downloads. Add an `<ignore type="sregex">` entry for suffixes such as `.crdownload` and `.tmp`.	Verify with a real download. Risk: an attacker could use an ignored suffix.
R2	Raise severity for executables and scripts	Make risky file types stand out. A local rule on 554 matching the path and extension (example below).	Verify by creating a dummy `.ps1`. Impact: more alerts to triage.
R3	Tune rule 553 in Downloads	Reduce false positives from routine deletes, e.g. lower level for non-executable files.	Verify alert levels afterwards. Risk: hiding malicious cleanup.
R4	Test a real browser download	Close the key evidence gap (S7).	Record process name and any temporary-file alerts.
R5	Confirm who-data after restarts	Always check 6019 and the audit fields.	Residual risk if SACL is changed later.
R6	Manage the setting centrally	Push the policy through the manager (`agent.conf`) for many endpoints.	Needs testing; wider blast radius if wrong.
R7	Snapshot before changes	Take a VirtualBox snapshot before editing configuration.	Storage cost only.

Example only (not tested): ignore entry inside `<syscheck>`

```
<ignore type="sregex">\.crdownload$|\.tmp$</ignore>
```

Example only (not tested): local rule in `local_rules.xml`

```
<group name="syscheck,">
  <rule id="100100" level="8">
    <if_sid>554</if_sid>
    <field name="file" type="pcre2">(?i)\\downloads\\.*\.(exe|msi|ps1|bat|cmd|js|vbs)$</field>
    <description>Executable or script added to a Downloads folder</description>
  </rule>
</group>
```

13. What I Learned

Concepts. I learned that FIM only sees what it is told to watch. Day 9 felt complete, but when I asked about my own Downloads folder the honest answer was "not until you add it". I also learned to separate the stages between an action and an alert; that separation is what let me interpret an empty dashboard calmly.

Practical skills. I backed up and edited `ossec.conf`, restarted the service, and used `Select-String` on the agent log to prove that monitoring had started before I trusted any test. I read the who-data fields to see user, process and PID, and compared hashes across three alerts to confirm the file's history.

Troubleshooting. My first search returned nothing. The answer was not to change the configuration but to create a known file and watch what happened. I also saw that I cannot always name the cause: I can say monitoring was working, but not exactly why nothing appeared at 10:46.

Evidence analysis. Two files with identical content had identical hashes, a concrete demonstration that a hash describes content, not identity. The same-second edit and delete taught me that a missing alert is not proof of no activity.

Detection engineering. A routine deletion scored level 7 with Defense Evasion and Data Destruction tags. I now see that default rules are written for the general case, and that a folder like Downloads needs tuning, which I have proposed but not tested.

Still to develop. I have not tested a real browser download, written and tested a custom rule, or resolved the 6021 warning.

14. Practical Knowledge and Competencies

Table 21 — Skills mapped to lab tasks

Skill	Lab task	Level shown
Endpoint configuration	Backed up and edited `ossec.conf`, restarted the agent.	Performed
Log verification	Searched the agent log for 6003, 6012, 6019.	Performed
Alert triage	Read rule, level, groups, MITRE tags and who-data fields.	Performed
Hash evidence analysis	Compared before and after hashes across alerts.	Performed
Limitation analysis	Identified the same-second gap and the noise.	Performed

Skill	Lab task	Level shown
Custom rule writing	Example rule drafted.	Studied only (not tested)
Browser download detection	None.	Not performed

14.1 Suggested next exercises

1. Download a small file with Edge, edit it in Notepad, delete it, and record the process names and any temporary-file alerts.
2. Apply and test the ignore entry and the custom rule.
3. Repeat the same-second test with `Start-Sleep` intervals of one, two and five seconds to see where events stop being collapsed.
4. Resolve warning 6021 and retest `FIM_Test`.

15. Reproduction Guide

15.1 Initial state

A working Wazuh manager and dashboard; a Windows 10 endpoint enrolled as an active agent; the Day 9 FIM configuration present; Administrator access. Take a VM snapshot first.

15.2 Setup

Table 22 — Commands to run and what to expect

#	Command (Windows VM, Administrator PowerShell)	Expected
1	`echo $env:USERPROFILE\Downloads`	Your Downloads path.
2	`Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" "C:\Program Files (x86)\ossec-agent\ossec.conf.bak2"`	No output; backup created.
3	`notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"`	Notepad opens. In `<syscheck>`, under `<frequency>`, add the line in Section 7.1 with your path. Check tags are balanced; save.
4	`Restart-Service -Name wazuh` then `Get-Service -Name wazuh`	Status Running.
5	`Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.log" -Pattern "downloads|whodata|real-time" | Select-Object -Last 10`	6003 for your path, 6012 and 6019.

15.3 Test activities

Run each command on its own and wait about ten seconds between them.

```
Get-Date; "download test" | Out-File "$env:USERPROFILE\Downloads\wazuh_test2.txt"
Get-Date; Add-Content "$env:USERPROFILE\Downloads\wazuh_test2.txt" "My Name is Hope"
Get-Date; Remove-Item "$env:USERPROFILE\Downloads\wazuh_test2.txt"
```

15.4 Verification and evidence collection

1. In the dashboard open Integrity monitoring, then Events, pick the agent, and search `syscheck.path: *wazuh_test2*` with a wide time range.
2. Expect three rows: added, modified, deleted. Open each and record rule ID and level, user, process, PID, sizes and hashes.
3. Capture the log lines from step 5 and each alert's detail view.

15.5 Troubleshooting

Table 23 — Common problems

Problem	What to do
Service will not start	Check the XML tags; restore the backup (Section 5.3).
No 6003 line for Downloads	The path may be wrong or the file unsaved; recheck step 3 and restart.
Alerts lack user and process	Who-data may be unavailable (see 6021); check 6019 and restart the agent.
"No results"	Widen the time range, remove narrow filters, and create a file directly in the folder.
Modified alert missing	Increase the pause between edit and delete.

15.6 Cleanup and rollback (documented, not executed in this lab)

The test files are removed by the test itself. To remove monitoring, delete the added line (or restore the backup) and restart the agent.

16. Limitations and Future Improvements

16.1 Limits of the experiment

- One endpoint, one folder, one user and one program (PowerShell) were tested.
- Only plain text files were used; no executables, archives or documents.
- No browser download was performed, which was the original motivating scenario.
- Only Wazuh was used as a data source, so findings about process activity rest on who-data alone.

16.2 Timestamp and display anomalies

The alert timestamps agree with the Windows clock: the commands ran at 11:19:33 and the deleted alert is stamped 11:19:34. The `syscheck.mtime_after` field, however, displays one hour *later* than the alert (Figure 26): 12:27:09 against an alert at 11:27:10.

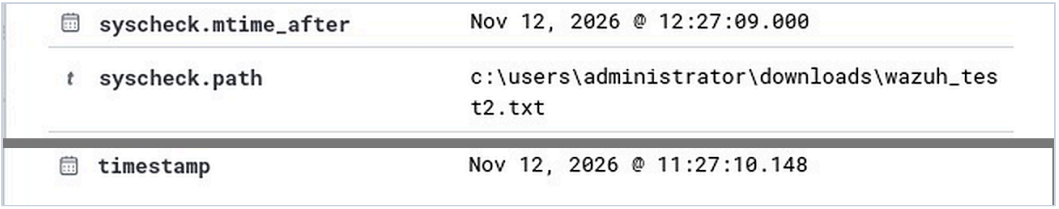


Figure 26 *— Two rows from the creation alert of wazuh_test2.txt. Upper panel: mtime_after 12:27:09. Lower panel: alert timestamp 11:27:10.148. Panels cropped from separate screenshots and joined; nothing altered.*

The modification alert's full log gives the earlier modification time as the Unix epoch value `1794479229` (Figure 17). Converted, that is 10:27:09 UTC, or 11:27:09 in Lagos time (UTC+1). That matches the alert time, so the real modification time agrees with the clock and the extra hour is most likely added by the

dashboard when it displays that field. This is a calculated inference, not a confirmed explanation, and it is consistent with the one-hour offset recorded in Day 9.

16.3 Evidence gaps

- No screenshot of the creation command for `wazuh_test.txt`, or of any `Get-Date` output for `wazuh_test2.txt`.
- Numeric rule ID for the creation of `wazuh_test.txt` and for its deletion not shown.
- The "after" SHA-256 of the modification alert not captured.
- The Wazuh version is not recorded.

16.4 False positives and false negatives

False positives: ordinary deletes and edits raise level 7 alerts with attack-technique tags; browser temporary files are expected to add more (untested). **False negatives:** the same-second case shows a change can go unreported; files saved outside the monitored path are invisible.

16.5 Future improvements

See Section 12.2 and 14.1: test browser downloads, apply the ignore and severity rules, explain 6021, and test sub-second and multi-file bursts.

17. Conclusion

This extension showed that the Day 9 technique carries over to the Windows Downloads folder with a single, reversible configuration line. The agent log proved the folder was watched in real time with who-data, and a controlled test file produced the expected sequence of rule 554, 550 and 553 alerts, each naming the user, the process and the file's fingerprints. The deleted-file rule ID missing from Day 9 was captured. The work also found limits: a same-second edit and delete left no modified alert; routine deletes carry alarming tags; and one who-data warning remains unexplained. The strongest result is a repeatable method; the most important gap is that a real browser download has not yet been tested. The conclusions here apply to PowerShell-created text files in one lab folder, nothing more.

Appendix A — Command History

Table 24 — Commands executed

#	Command	Purpose
1	`echo $env:USERPROFILE\Downloads`	Get the path
2	`Copy-Item "...\ossec.conf" "...\ossec.conf.bak2"`	Backup
3	`notepad "...\ossec.conf"`	Edit configuration
4	`restart-service -name wazuh`	Apply configuration

#	Command	Purpose
5	`get-service -name wazuh`	Confirm Running
6	`Select-String ... -Pattern "downloads|whodata|real-time" | Select-Object -Last 10`	Verify monitoring
7	`Get-Date; "download test" | Out-File ...\wazuh_test.txt`	Test 1 (command screenshot not captured)
8	`Get-Date; Add-Content ...\wazuh_test.txt "My Name is Hope"` and `Get-Date; Remove-Item ...\wazuh_test.txt`	Same-second case
9	Create, `Add-Content`, `Remove-Item` on `wazuh_test2.txt`	Clean lifecycle

Appendix B — Configuration and Log Excerpts

```
<frequency>43200</frequency>
<directories check_all="yes" realtime="yes" whodata="yes"
report_changes="yes">C:\FIM_Test</directories>
<directories check_all="yes" realtime="yes"
whodata="yes">C:\Users\Administrator\Downloads</directories>
```

Table 25 — Alert IDs read from screenshots

File	Event	Alert ID
`wazuh_test.txt`	added	`1794477288.768442`
`wazuh_test2.txt`	added	`1794479230.886952`
`wazuh_test2.txt`	modified	`1794479300.888339` (partly visible in the screenshot)

Appendix C — Evidence Register

Table 26 — Evidence register

File	Content	Used as
WA0097	Commands and Notepad with both directories lines	Figure 3
WA0098	Restart, service status, agent log	Figure 4, Figure 5
10:46 photo	Dashboard "No results"	Figure 6
WA0099	Events list, test 1 added	Figure 7
WA0100, 0102, 0105	Test 1 alert hashes and timestamp	Omitted (duplicates of Figure 11)
WA0101, 0103, 0104, 0106	Test 1 alert details	Figure 9, Figure 8, Figure 10, Figure 11
WA0107	Same-second commands	Figure 12
WA0125	Events list, all file events	Figure 13

File	Content	Used as
WA0108, 0113, 0114	Test 2 creation alert	Figure 14, Figure 15, Figure 16
WA0109, 0111	Test 2 creation: mtime and timestamp	Figure 26
WA0112, 0118, 0120, 0122	Test 2 modification alert	Figure 17, Figure 19, Figure 20, Figure 18
WA0116, 0124	Modification alert repeats	Omitted (duplicates)
WA0115, 0117, 0119, 0123	Test 2 deletion alert	Figure 21, Figure 22, Figure 23, Figure 24
WA0121	Deletion alert hashes	Omitted (duplicate of Figure 24)

Appendix D — Glossary

Table 27 — Glossary

Term	Meaning
Agent	Wazuh software on the endpoint that collects and sends events.
Alert	An event that matched a rule and was stored with a severity level.
Hash	A fingerprint calculated from a file's contents.
SACL	System access control list: Windows audit settings placed on a folder.
syscheck	The Wazuh module that performs FIM.
Who-data	User and process information attached to a FIM event through auditing.
PID	Process ID: the number identifying a running program instance.
DQL	The dashboard query language used in the Events search bar.

Appendix E — References

- Day 9 report: File Integrity Monitoring with Wazuh (this lab's baseline).
- Wazuh documentation: File integrity monitoring, and the who-data audit option. documentation.wazuh.com
- MITRE ATT&CK: T1565.001 Stored Data Manipulation; T1070.004 File Deletion; T1485 Data Destruction. attack.mitre.org
- PCI DSS requirement 11.5; NIST SP 800-53 control SI-7.